Table of Contents

1. Executive Summary 2

2. Commit Review: What Changed on the Branch 2

3. Live Deployment Audit: Observed Symptoms 4

4. Root Cause: The max-w Token Collision 6

4.1 How Tailwind v3 resolved these utilities . . . 6

4.2 What changed under Tailwind v4 . . . 6

5. Blast Radius Assessment 8

6. Fix Verification: Minimal Reproduction 9

7. Secondary Migration Risks 9

8. Recommended Remediation Plan 10

8.1 Immediate hotfix (restores the interface) . . . 10

8.2 Short-term hardening . . . 11

8.3 Prevention . . . 11

9. Appendix: Evidence Index 11

1. Executive Summary

The self-hosted Docker deployment of the Clara AI Platform at clara.ai.amazetechclans.com regressed visually after the latest dependency updates were pulled from the branch feature/ai-core-from-doable-source. The login card collapsed to a width of 8 pixels, the landing page hero subtitle renders as a vertical stack of single words, the signup card measures 24 pixels wide, and text blocks across pricing, contact, docs, and features pages are squeezed into unreadable narrow columns. This report confirms that the regression is systemic rather than page-specific: every container that uses a max-w-xs through max-w-3xl utility is affected across the platform.

The root cause is a semantic collision introduced by the Tailwind CSS v3 to v4 migration performed on the branch between October 8 and October 9, 2026. The legacy tailwind.config.js file defines a custom named spacing scale (xs, sm, md, lg, xl, 2xl, 3xl) that resolves to very small values (0.25rem to 4rem). Under Tailwind v3 these names were harmless because max-width utilities drew from a separate maxWidth theme namespace. Under Tailwind v4, loaded through the @config compatibility directive, those same names become spacing theme tokens that shadow the container scale, so max-w-sm now compiles to max-width: var(--spacing-sm), an effective width of 8px instead of the expected 384px.

The fix is minimal and provably safe: delete the unused spacing block from tailwind.config.js. A minimal reproduction built for this report confirms that removing the block restores max-w-* resolution to the container scale (max-w-sm returns to 24rem). The named spacing utilities that the block would generate, such as p-sm or gap-md, have zero usages in the source tree, so the deletion carries no regression risk. Chapter 8 provides the full remediation plan, including secondary Tailwind v4 and Next.js 16 behavioral changes that should be audited after the hotfix ships.

97	57	8px	1 block
broken utility usages	source files affected	login card rendered width	config deletion to fix

2. Commit Review: What Changed on the Branch

The branch feature/ai-core-from-doable-source advanced from the reviewed baseline 88a6b5f9 to the new head 2c61afdc, a window of 1,995 commits authored between October 4 and October 10, 2026. Nearly half of the volume is governance logging in docs(project-control), reflecting the team's practice of recording every working session. The functional work clusters into three themes: the WhatsApp lifecycle implementation, the Doable source vendoring and AI platform core extraction, and a series of page-level user interface fixes for card clipping and overflow symptoms that appeared after the dependency migration.

Commit type	Count	Share	Primary theme
docs	981	49.2%	Governance logs, project-control records
fix	403	20.2%	WhatsApp lifecycle, UI clipping band-aids
test	174	8.7%	Route-level regression coverage
feat	161	8.1%	WhatsApp lifecycle, agent features
chore / ops / build / ci	76	3.8%	Docker, lockfile, pipeline work
refactor / security / perf / worklog	46	2.3%	Miscellaneous hygiene

Table 2-1. Composition of the 1,995 new commits since baseline 88a6b5f9.

The dependency event the deployment team remembers as the trigger is commit 4688c938 on October 6, titled "Implement feature X to enhance user experience and optimize performance", which regenerated package-lock.json with 127,254 insertions and 9,441 deletions. That regeneration floated every caret-ranged dependency but did not by itself break the interface, because tailwindcss was still pinned to 3.4 at that point. The breaking change arrived two days later inside commit a126fa19, the Doable source vendoring commit, which upgraded the styling and framework majors in a single step.

Date (2026)	Commit	Change
Oct 6, 15:53	4688c938	package-lock.json regenerated: +127,254 / -9,441 lines (all caret-ranged deps floated)
Oct 8, 01:31	a126fa19	next 15.5.24 to 16.4.0; tailwindcss 3.4 to 4.3; tailwind-merge 2.6.0 to 3.7.0; autoprefixer and postcss bumped
Oct 9, 02:08	f6647f04	postcss.config.js switched from the v3 tailwindcss plugin to @tailwindcss/postcss
Oct 9, 02:09	40141867	Reverted two minutes later: the v4 PostCSS plugin was not yet present in the lockfile
Oct 9, 02:19	d119c028 / 26d60820	PostCSS v4 plugin re-applied and @tailwindcss/postcss 4.3.3 declared in package.json
Oct 9, 02:20	0d1666ef	Lockfile entries added for the v4 toolchain (tailwindcss, oxide, node bindings)
Oct 9, 03:47	449afb73	globals.css rewritten from @tailwind directives to @import "tailwindcss" plus @config "../../tailwind.config.js"
Oct 10, 03:55-04:05	12 fix(ui) / test(ui) commits	Page-level band-aids: min-w-0, max-w-full, box-border, overflow-wrap added to pricing, checkout, contact, signup, and password-reset screens

Table 2-2. Dependency-migration timeline that produced the styling regression.

The sequence in Table 2-2 shows a migration performed under time pressure: the PostCSS configuration was applied, reverted, and re-applied within eleven minutes, and the stylesheet entry was converted the following hour. No commit in the window audits what the new Tailwind v4 engine would do with the existing v3-era tailwind.config.js theme, and that omission is precisely where the regression hides.

3. Live Deployment Audit: Observed Symptoms

The deployed site was audited with an automated browser session at desktop (1440 x 900) and mobile (390 x 844) viewports. For every public page, two kinds of evidence were collected: full-page screenshots, and direct DOM geometry measurements of the elements carrying max-w-* classes. The deployed stylesheet was also downloaded and inspected rule by rule. The running deployment matches the branch head: the login screen markup served in production is byte-for-byte the markup of login-screen.tsx at 2c61afdc, including its band-aid classes, so the findings below describe the current branch state and not a stale build.

Page	Collapsed element	Utility	Expected (v3)	Rendered (live)
Landing	Hero subtitle paragraph	max-w-2xl	42rem (672px)	48px
Landing	Demo section container	max-w-3xl	48rem (768px)	64px
Landing	Closing CTA container	max-w-3xl	48rem (768px)	114px (clamped by child)
Login	Authentication card	max-w-sm	24rem (384px)	8px
Pricing	FAQ container	max-w-2xl	42rem (672px)	48px
Contact	Header paragraph	max-w-xl	36rem (576px)	32px
Signup	Account-creation card	max-w-lg	32rem (512px)	24px
Docs	Section intro paragraph	max-w-2xl	42rem (672px)	48px
Features	Section paragraph	max-w-2xl	42rem (672px)	48px
Signup (mobile)	Document body horizontal overflow	-	no overflow	418px vs 390px viewport

Table 3-1. Measured DOM geometry of collapsed containers on the live deployment.

The login page is the most dramatic case and is worth describing precisely, because it demonstrates that the measurements are not a rendering fluke. The heading element "Welcome to Clara AI" renders at 8 pixels wide and 134 pixels tall, which forces the browser to wrap the text word by word into a vertical strip. The login form itself measures 0 pixels wide inside a flex row, and each input field renders as a 34-pixel-wide slit. The ancestor chain confirms the collapse originates at the card container, whose class list reads "w-full min-w-0 max-w-sm": the width and minimum-width utilities resolve normally, and the max-w-sm utility clamps the card to 8 pixels.

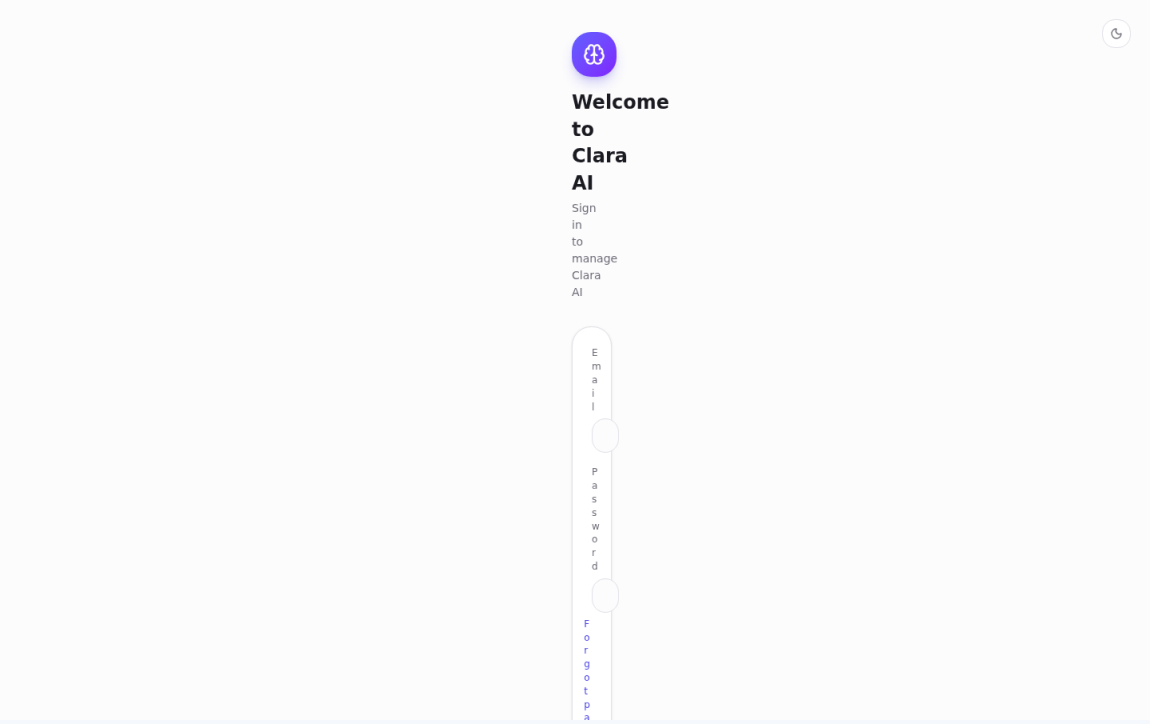


Figure 3-1. Live login page at 1440 x 900: the auth card is collapsed to 8px, text wraps vertically, and inputs render as slits.

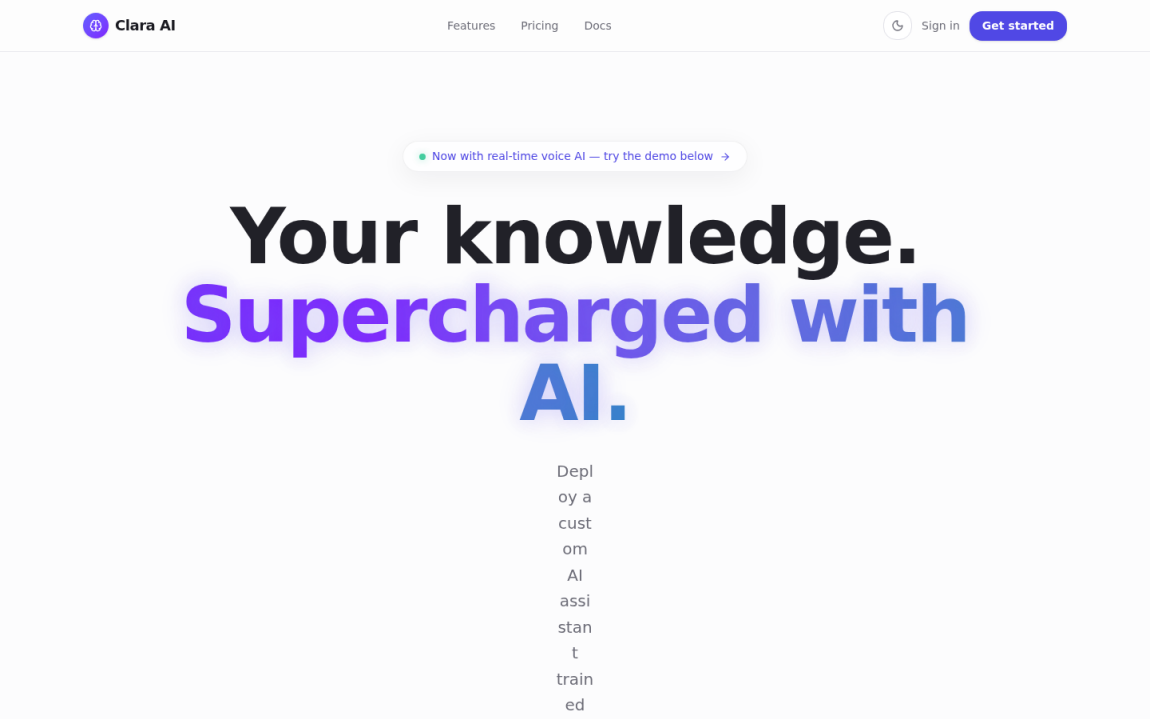


Figure 3-2. Live landing hero: the max-w-2xl subtitle wraps into a 48px-wide word stack beneath the headline.

At the mobile viewport the signup page additionally pushes the document to a scrollWidth of 418 pixels against a 390-pixel viewport, producing a 28-pixel horizontal overflow. That overflow is the browser refusing to compress unbreakable content, such as button labels and email inputs, below their minimum content size inside a 24-pixel-wide card. The pattern matches the wave of fix(ui) commits from October 10,

which patched pricing, checkout, contact, and password-reset screens with min-w-0 and overflow-wrap constraints; those patches treat the symptom per page while the systemic utility resolution defect remains untouched in the build.

4. Root Cause: The max-w Token Collision

4.1 How Tailwind v3 resolved these utilities

Under Tailwind CSS 3.4, the engine that styled this platform until October 8, max-width utilities and spacing utilities read from two independent theme namespaces. The maxWidth namespace defined max-w-xs as 20rem, max-w-sm as 24rem, and so on up to max-w-3xl at 48rem. The spacing namespace governed padding, margin, gap, width, and height values. A project could therefore define custom spacing keys named sm or xl without ever touching the meaning of max-w-sm, and the repository did exactly that: its design-token configuration mapped a handful of named spacing keys to CSS variables for a token pipeline that ultimately was never consumed.

4.2 What changed under Tailwind v4

Tailwind v4 replaced the JavaScript-first theme with a CSS-first theme in which utilities resolve named tokens dynamically against the --spacing-* and --container-* theme scales. When the v4 engine loads a legacy configuration through the @config directive, every theme.spacing entry is promoted to a real --spacing-* theme token. The migration commits kept the old tailwind.config.js untouched, so its named spacing keys xs through 3xl became first-class spacing tokens, and those names now take precedence over the container scale when the engine compiles max-w-* utilities. The result is that max-w-sm emits max-width: var(--spacing-sm) instead of max-width: var(--container-sm).

```
// tailwind.config.js (lines 130-137, unchanged since the v3 era)
spacing: {
  xs: "var(--spacing-xs)",
  sm: "var(--spacing-sm)",
  md: "var(--spacing-md)",
  lg: "var(--spacing-lg)",
  xl: "var(--spacing-xl)",
  "2xl": "var(--spacing-2xl)",
  "3xl": "var(--spacing-3xl)",
}
```

Listing 4-1. The legacy named spacing scale that collides with v4 container tokens.

```
/* src/app/globals.css (lines 116-122) - token values */
:root {
  --spacing-xs: 0.25rem;   /*  4px */
  --spacing-sm: 0.5rem;    /*  8px */
  --spacing-md: 1rem;      /* 16px */
  --spacing-lg: 1.5rem;    /* 24px */
  --spacing-xl: 2rem;      /* 32px */
  --spacing-2xl: 3rem;     /* 48px */
  --spacing-3xl: 4rem;     /* 64px */
}
```

Listing 4-2. The CSS variable values the spacing tokens resolve to.

```
/* Served in production: /_next/static/css/1a542fd84dc40983.css */
.max-w-xs  { max-width: var(--spacing-xs) }
.max-w-sm  { max-width: var(--spacing-sm) }
.max-w-md  { max-width: var(--spacing-md) }
.max-w-lg  { max-width: var(--spacing-lg) }
.max-w-xl  { max-width: var(--spacing-xl) }
.max-w-2xl { max-width: var(--spacing-2xl) }
.max-w-3xl { max-width: var(--spacing-3xl) }
.max-w-4xl { max-width: var(--container-4xl) }   /* correct: 56rem */
```

Listing 4-3. Deployed utility rules proving the token shadowing in production.

Utility	Deployed rule	Effective width	Tailwind v3 width
.max-w-xs	`var(--spacing-xs)`	0.25rem = 4px	20rem = 320px
.max-w-sm	`var(--spacing-sm)`	0.5rem = 8px	24rem = 384px
.max-w-md	`var(--spacing-md)`	1rem = 16px	28rem = 448px
.max-w-lg	`var(--spacing-lg)`	1.5rem = 24px	32rem = 512px
.max-w-xl	`var(--spacing-xl)`	2rem = 32px	36rem = 576px
.max-w-2xl	`var(--spacing-2xl)`	3rem = 48px	42rem = 672px
.max-w-3xl	`var(--spacing-3xl)`	4rem = 64px	48rem = 768px
.max-w-4xl - .max-w-7xl	`var(--container-*)`	unchanged	unchanged

Table 4-1. Token resolution on the deployed stylesheet versus the previous v3 behavior.

Two details in Table 4-1 explain why the breakage feels selective rather than total. First, the container utilities max-w-4xl through max-w-7xl are unaffected, because the legacy spacing scale stops at 3xl; pages whose outermost wrappers use max-w-5xl, such as the patched contact and pricing screens, therefore look superficially healthy even while their inner max-w-xl paragraphs collapse. Second, the shadcn-style color tokens (bg-background, text-muted-foreground, border-border) all resolve correctly through the @config compatibility layer, so colors, buttons, and typography look right, which masks the layout defect until a container width is actually measured.

5. Blast Radius Assessment

The defect is not confined to the public marketing pages. A word-boundary search across the source tree finds 97 usages of the seven broken utilities spread over 57 files, including authenticated admin screens, the agent workspace, the embeddable widget, and the voice screens. Any user flow that passes through one of these containers on the current build inherits the collapse. The per-utility distribution and the most heavily affected files are tabulated below.

Utility	Usages	Effective width on live build
max-w-xs	8	4px
max-w-sm	12	8px
max-w-md	22	16px
max-w-lg	9	24px
max-w-xl	10	32px
max-w-2xl	25	48px
max-w-3xl	15	64px

Table 5-1. Distribution of the 97 broken utility usages across the source tree.

File (top consumers)	Usages
src/features/agent-conversation/components/conversation-chat-window.tsx	7
src/features/embed/components/DemoWebsiteFrame.tsx	6
src/components/admin/ai-platform/AiPlatformWorkspace.tsx	5
src/components/admin/ScrapeWizard.tsx	5
src/features/platform/screens/landing-screen.tsx	4
src/features/embed/screens/voice-screen.tsx	3

Table 5-2. Files with the highest concentration of broken max-w utilities.

The decisive safety observation for the fix is that the named spacing scale itself is dead configuration. A search for any utility that would consume those tokens, including p-sm, m-lg, gap-md, w-xl, and their padding, margin, width, and height relatives, returns zero matches across the entire source tree. The production stylesheet confirms the same fact from the compiler side: none of .p-sm, .gap-md, or .m-lg is emitted at all. The block generates nothing the application uses and exists only to poison seven max-width utilities, which makes its removal uniquely low-risk among the possible remediations.

6. Fix Verification: Minimal Reproduction

To move the analysis from inference to proof, a minimal reproduction was built with the exact production toolchain versions: tailwindcss 4.3.3, @tailwindcss/postcss 4.3.3, and postcss 8.5.6. The reproduction mirrors the production pipeline: a stylesheet that imports tailwindcss and loads the JavaScript configuration through @config, a :root block carrying the same spacing variable values as globals.css, and a content file that uses the affected utilities. The generator was then run twice, once with the legacy spacing block present and once with it deleted.

```
/* build.js output - WITH the legacy spacing block (current repo state) */
.max-w-sm  { max-width: var(--spacing-sm); }    /* resolves to 0.5rem = 8px */
.max-w-xl  { max-width: var(--spacing-xl); }    /* resolves to 2rem   = 32px */
.max-w-2xl { max-width: var(--spacing-2xl); }   /* resolves to 3rem   = 48px */

/* build.js output - WITHOUT the spacing block (the fix) */
.max-w-sm  { max-width: var(--container-sm); }  /* resolves to 24rem = 384px */
.max-w-xl  { max-width: var(--container-xl); }  /* resolves to 36rem = 576px */
.max-w-2xl { max-width: var(--container-2xl); } /* resolves to 42rem = 672px */
```

Listing 6-1. Minimal reproduction output, before and after removing the spacing block.

Variant	.max-w-sm compiles to	Effective width
Legacy spacing block present (current branch)	max-width: var(--spacing-sm)	0.5rem = 8px
Spacing block deleted (fix)	max-width: var(--container-sm)	24rem = 384px

Table 6-1. Reproduction matrix proving the fix restores v3 sizing semantics.

The reproduction confirms the causal chain end to end: the presence of the named spacing scale alone flips the emitted rule from the container scale to the spacing scale, and its absence restores the exact widths the interface was designed against. Because zero source files consume the named spacing utilities, deleting the block changes nothing else in the generated stylesheet. The reproduction script and both generated stylesheets are preserved under scripts/tw4-repro/ for independent verification.

7. Secondary Migration Risks

The token collision is the primary defect, but the same migration introduced several second-order behavioral changes that the team should audit once the hotfix ships. Each item below was verified directly against the deployed stylesheet or the source tree, with usage counts included where they quantify exposure.

Risk	Evidence (live build)	Impact
Default border color is now currentColor	.border emits border-style and border-width only; v3 preflight set gray-200	Bare `border` elements render dark, text-colored outlines instead of subtle gray
container class lost centering and padding	.container emits width:100% plus max-widths only; v3 config had center:true, padding:2rem	2 usages render full-bleed and left-aligned
space-y / space-x direction flip	Deployed selector is :not(:last-child) with margin-block-end; v3 used margin-top on * + *	424 usages; spacing can migrate to the wrong side when children carry their own margins
Shadow and radius scale renames	v4 shifted the named scale (old shadow-sm now shadow-xs, rounded-sm now rounded-xs)	Depths and radii render one step larger than designed
outline-none semantic split	v4 renames the old invisible-outline behavior to outline-hidden	Focus rings may appear or vanish unexpectedly on focusable elements
autoprefixer runs after Lightning CSS	postcss.config.js still lists autoprefixer beside @tailwindcss/postcss	Redundant double prefixing; larger CSS with no visual gain
Modern browser baseline	v4 targets Safari 16.4+, Chrome 111+, Firefox 128+ via Lightning CSS	Older browsers may drop cascade-layer or color-mix rules

Table 7-1. Secondary Tailwind v4 and Next.js 16 behavioral changes verified on the deployment.

The border-color change deserves first attention after the hotfix, because it is the most visually widespread of the secondary risks. Under v3 the preflight assigned a light gray to every border, so a class list reading simply "border" produced a subtle outline; under v4 the same element inherits its text color as the border color, which on dark headings and buttons produces heavy outlines that read as visual defects. The audit order should run: bare border usages, the two container usages, then a spot check of space-y sequences that mix in explicit margins.

8. Recommended Remediation Plan

8.1 Immediate hotfix (restores the interface)

The hotfix is the deletion of the seven-entry spacing object from tailwind.config.js, followed by a dependency install to keep the lockfile consistent, a full Docker rebuild, and a redeploy. The verification gate should be mechanical rather than impressionistic, because this regression slipped through exactly due to a missing mechanical check: after the rebuild, fetch the served stylesheet and assert that .max-w-sm contains var(--container-sm), then measure the login card bounding box at 1440 pixels wide and confirm a width of at least 384 pixels.

Step	Action	Pass criterion
1	Delete the spacing block (Listing 4-1) from tailwind.config.js	grep 'spacing:' tailwind.config.js returns no theme entries
2	Run npm install to sync package-lock.json	Lockfile diff shows no unrelated churn
3	Rebuild and redeploy the Docker image	Deployment serves a new CSS hash
4	Assert served CSS token resolution	.max-w-sm uses var(--container-sm)
5	Measure the login card on the live site	Card width >= 384px at 1440px viewport
6	Measure mobile signup overflow	scrollWidth <= 390px at 390px viewport

Table 8-1. Hotfix procedure with mechanical verification gates.

8.2 Short-term hardening

Three follow-ups close the remaining exposure from the same migration. First, sweep the 57 affected files for the secondary risks in Table 7-1, starting with bare border classes, which should gain explicit border colors or a base-layer default. Second, remove autoprefixer from postcss.config.js, since the v4 engine already emits vendor-prefixed output through Lightning CSS and the duplicated pass only inflates the bundle. Third, re-evaluate tailwindcss-animate, which the legacy config loads as a plugin; v4 projects conventionally replace it with tw-animate-css, and the legacy plugin path through @config is a compatibility surface the team should not carry long term.

8.3 Prevention

Two guardrails would have caught this class of defect before production. The first is a post-build stylesheet assertion in CI: after next build, grep the emitted CSS for the known-good resolution of sentinel utilities (max-w-sm, p-4, rounded-lg) and fail the pipeline if a sentinel resolves to an unexpected token. The second is a lightweight visual smoke suite using headless browser screenshots of the login, landing, and signup pages with element-width assertions on the primary containers; the existing Playwright base image in the Dockerfile means the runtime for such a suite is already present in the build graph. Finally, major dependency upgrades of this size deserve their own isolated commit and review, separate from a large vendoring change, so that a bisect between the working and broken builds lands on a single intentional diff.

9. Appendix: Evidence Index

All evidence cited in this report is preserved on disk for independent verification. Screenshots were captured from the live deployment on October 10, 2026, at the head commit 2c61afdc; the reproduction artifacts were generated with the exact production dependency versions; and the served stylesheet was downloaded directly

from the deployment origin. The table below maps each artifact to its role in the analysis.

Artifact	Role
`download/ui-audit/login-desktop.png`	Figure 3-1 source; collapsed login card at 1440x900
`download/ui-audit/landing-desktop-view.png`	Figure 3-2 source; hero subtitle word-stacking
`download/ui-audit/signup-mobile.png`	Mobile signup overflow evidence (418px vs 390px)
`download/ui-audit/*.png (12 files)`	Desktop and mobile captures of all audited pages
`scripts/tw4-repro/build.js`	Minimal reproduction generator (with/without spacing variants)
`scripts/tw4-repro/out-with-spacing.css`	Reproduced broken token resolution
`scripts/tw4-repro/out-no-spacing.css`	Proof of fix: container-scale resolution restored

Table 9-1. Evidence artifact index.

Repository coordinates for the analysis: repository Rilan-Dev/Clara-AI-Platform, branch feature/ai-core-from-doable-source, baseline commit 88a6b5f9, reviewed head 2c61afdc, deployment origin clara.ai.amazetechclans.com. The deployed stylesheet audited in Chapter 4 is served at /_next/static/css/1a542fd84dc40983.css, and the DOM measurements in Chapter 3 were taken with the browser positioned at the viewports stated in each figure caption.